e41de89c-1a37-4a6f-9c93-32afd75424

<current>

<request>

You are working on an issue in the 'SpudGunMan/node-slurper' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/node-slurper/node-slurper (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-10-21T18:12:43.622Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section details on the original issue you should resolve

<issue_title>.ino file enhance with decrypting packets</issue_title>
<issue_description>Complete this as fully working example to display data from UDP.

https://github.com/SpudGunMan/node-slurper/blob/main/meshview.ino

Currently decodes outer wrapper but has no decryption for the keys.

Haven't tested but assume licensed mode will decrypt as it's clear text as well</issue_description>

Comments on the Issue (you are @copilot in this section)

<comments>

<comment_new><author>@SpudGunMan</author>
Some details on encryption

https://meshtastic.org/docs/overview/encryption/

https://meshtastic.org/docs/about/overview/encryption/
limitations/

If any improvements otherwise are seen to handling packets.

https://meshtastic.org/docs/overview/mesh-algo/

</comment_new>
<comment_new><author>@SpudGunMan</author>
Encryption in the Meshtastic firmware is handled primarily through the `CryptoEngine` class and its platform-specific implementations (such as `ESP32CryptoEngine`). Here’s where encryption is managed in the codebase

https://github.com/meshtastic/firmware

---

1. src/mesh/CryptoEngine.h & src/mesh/CryptoEngine.cpp

- The `CryptoEngine` class provides the main interfaces for encryption and decryption, such as:
  - `encryptPacket(...)`
  - `decrypt(...)`
  - `encryptAESCtr(...)` (AES in counter mode)
  - `encryptCurve25519(...)` and `decryptCurve25519(...)` (for PKI/Curve25519 encryption)
- Example implementation:
  ```cpp
  void CryptoEngine::encryptPacket(uint32_t fromNode, uint64_t packetId, size_t numBytes, uint8_t *bytes) {
      if (key.length > 0) {
          initNonce(fromNode, packetId);
          if (numBytes <= MAX_BLOCKSIZE) {
              encryptAESCtr(key, nonce, numBytes, bytes);
          } else {
              LOG_ERROR("Packet too large for crypto engine: %d. noop encryption!", numBytes);
          }
      }
  }
  ```

```

---

**2. Platform-specific implementation:**
- For the ESP32, the class `ESP32CryptoEngine` in `src/platform/esp32/ESP32CryptoEngine.cpp` implements the crypto functions using mbedtls:
  ```cpp
  virtual void encryptAESCtr(CryptoKey _key, uint8_t *_nonce, size_t numBytes, uint8_t *bytes) override {
      if (_key.length > 0) {
          mbedtls_aes_setkey_enc(&aes, _key.bytes, _key.length * 8);
          ...
          mbedtls_aes_crypt_ctr(&aes, numBytes, &nc_off, _nonce, stream_block, scratch, bytes);
      }
  }
  ```

---

3. Usage in Packet Handling (src/mesh/Router.cpp)
- When a packet is about to be sent, the function `perhapsEncode()` encrypts the data using the active channel key:
  ```cpp
  crypto->encryptPacket(getFrom(p), p->id, numbytes, bytes);
  memcpy(p->encrypted.bytes, bytes, numbytes);
  ```

---

4. Curve25519/PKI Encryption
- `encryptCurve25519` and `decryptCurve25519` support public key encryption for secure communications using node public keys.

Summary:
- Encryption is abstracted by the `CryptoEngine` class, with actual cryptographic routines (AES, Curve25519, etc.) implemented in both platform-agnostic (`src/mesh/CryptoEngine.cpp`) and platform-specific files (like `src/platform/esp32/ESP32CryptoEngine.cpp`).
  ```

```
- Packet encryption is triggered in routing code before sending
data over the mesh.
- The AES key and nonce generation are managed per-packet,
ensuring unique encryption for each transmission.</comment_new>
<comment_new><author>@SpudGunMan</author>
Something like this?

#include </comment_new>
<comment_new><author>@SpudGunMan</author>
Details on some aes arduino capability

https://www.arduinolibraries.info/libraries/aes-lib

And

https://www.arduinolibraries.info/libraries/aes-utils


As well as some others

https://github.com/DavyLandman/AESLib

https://www.arduinolibraries.info/libraries/aes_cmac


Having clear direction of which module to use which is supported
by the Arduino IDE
</comment_new>
</comments>

</problem_statement>
```

```
Implement the necessary changes to the repository so that the
requirements specified in the <problem_statement> are met.



Ensure that you lint, build and test early and iteratively to
validate each code change thoroughly.
```